Questionário investigativo do projeto integrador Full Stack

Identificação

Disciplina: Laboratório de Programação Full Stack

Data: 05/11/2026 Turma: Engenharia de Software Grupo: 6º Periodo

Integrante	Matrícula
Matheus Prates Gaspar Silva	202411269
2.	
3.	
4.	

Regra de resposta

Cada resposta deve apresentar: **ação executada**, **evidência coletada**, **arquivo e função localizados** e **explicação da relação entre o código e o resultado**. Respostas apenas conceituais não atendem à atividade.

Parte 1 Inicialização e arquitetura

1. Inicie a aplicação e registre a mensagem do terminal que prova que o servidor está disponível. Localize no comando executado o módulo e o objeto carregados pelo Uvicorn.

Ação executada: Executei `python -m uvicorn backend.app.main:app --port 8001`; o terminal informou “Uvicorn running on http://127.0.0.1:8001” e “Application startup complete”.

Evidência: Servidor respondeu em 200 na interface e no /docs. A instância já existente ocupava 8000; esta cópia foi executada em 8001 para isolar o exercício.

Arquivo e função: `backend/app/main.py`: objeto `app` (FastAPI) e `lifespan`; comando Uvicorn: módulo `backend.app.main`, objeto `app` (`modulo:objeto`).

Explicação: O Uvicorn importa `backend.app.main:app` e passa a servir o objeto FastAPI. O `lifespan` executa `criar_tabela()` antes do `yield`; saída textual registrada no log de comandos.

2. Abra a interface e o Swagger. Registre os dois endereços e explique por que ambos pertencem à mesma aplicação.

Ação executada: Abri a interface e a documentação Swagger no navegador local.

Evidência: Interface: http://127.0.0.1:8001/ (200). Swagger: http://127.0.0.1:8001/docs (200).

Arquivo e função: `backend/app/main.py`: `abrir_interface()` serve `frontend/index.html`; `app = FastAPI(...)` fornece as rotas e a documentação `/docs`.

Explicação: São caminhos da mesma instância `app`: `/` entrega o HTML e `/docs` gera a documentação interativa automaticamente a partir das rotas registradas.

3. Localize o `lifespan`. Que instrução ocorre antes do `yield`? Mostre no código onde a tabela é criada e explique por que reiniciar não apaga os registros.

Ação executada: Inspecionei `lifespan` e `criar_tabela` e reiniciei o servidor com uma reserva criada.

Evidência: `main.py`, linhas 14–17: `criar_tabela()` antes do `yield`. `database.py`, `CAMINHO_BANCO = .../dados/reservas.db` e SQL `CREATE TABLE IF NOT EXISTS`. Registro id 1 permaneceu após reinício.

Arquivo e função: `backend/app/main.py`: `lifespan`; `backend/app/database.py`: `conectar()` e `criar_tabela()`; `backend/app/services.py`: `criar_reserva()`.

Explicação: A inicialização garante que a tabela exista. Os dados ficam no arquivo SQLite persistente `dados/reservas.db`; `CREATE TABLE IF NOT EXISTS` não recria nem apaga a tabela quando ela já existe. O SQL `INSERT INTO reservas` grava a reserva.

4. Desenhe o fluxo de uma criação desde o clique em Salvar até a gravação no SQLite. Para cada etapa, informe o arquivo correspondente.

Ação executada: Acompanhei o envio pela interface e localizei cada etapa no código.

Evidência: Fluxo executado com POST 201 e JSON contendo id 2 na criação pela interface.

Arquivo e função: `frontend/index.html` (formulário); `frontend/app.js`: listener `submit`; `main.py`: `criar_reserva`; `schemas.py`: `ReservaEntrada`; `services.py`: `criar_reserva`; `database.py`: `conectar`.

Explicação: Clique em Salvar → evento `submit` → `fetch` POST `/api/reservas` com JSON → FastAPI valida `ReservaEntrada` → endpoint chama serviço → serviço verifica conflito e executa `INSERT INTO reservas` via SQLite → endpoint responde 201 e `lerResposta` atualiza mensagem, lista e resumo.

Parte 2 Contrato da API

5. No Swagger, identifique método, URL, corpo JSON, resposta e código HTTP da criação. Relacione cada item ao conceito de contrato da API.

Ação executada: Consultei no Swagger a operação `POST /api/reservas` e seu corpo/resposta documentados.

Evidência: Método POST; URL `/api/reservas`; corpo JSON: {"laboratorio":"Laboratório Evidência 6","data_reserva":"2026-11-05","turno":"noite","responsavel":"Matheus Prates Gaspar Silva","quantidade_pessoas":6,"finalidade":"Atividade investigativa Full Stack"}. Resposta observada: 201 e objeto equivalente com `id: 1`.

Arquivo e função: `backend/app/main.py`: decoração de `criar_reserva`; `schemas.py`: `ReservaEntrada` (entrada) e `ReservaSaida` (saída).

Explicação: O método e a URL identificam a operação; o corpo define os campos aceitos; `ReservaEntrada` valida o formato e limites; `ReservaSaida` acrescenta o id; 201 informa que um recurso foi criado. Esses elementos formam o contrato que cliente e servidor seguem.

6. Crie uma reserva válida pela interface. Registre a requisição no painel Rede do navegador ou repita a operação no Swagger. Localize a chamada `fetch`, a rota FastAPI e o modelo Pydantic envolvidos.

Ação executada: Preenchi o formulário da interface com Laboratório Interface, data 05/11/2026, turno noite, responsável Matheus Prates Gaspar Silva, quantidade 6 e finalidade de evidência; cliquei Salvar.

Evidência: A interface mostrou “Sucesso: reserva 2 salva.”; resumo passou a total 2, noite 2 (registro id 2 na base de exercício). A rota retornou 201.

Arquivo e função: `frontend/app.js`: handler `submit` monta `dadosDoFormulario()` e envia `fetch`; `backend/app/main.py`: `criar_reserva`; `backend/app/schemas.py`: `ReservaEntrada`.

Explicação: O handler envia JSON para POST `/api/reservas`; o FastAPI converte o corpo no modelo Pydantic, valida os campos e entrega os dados ao serviço. A resposta 201 é processada pelo frontend, que exibe sucesso e chama `listar()` para atualizar tabela e resumo.

7. Explique por que a API devolve `201` e não `200` na criação. Localize a configuração desse código no backend e o teste que o verifica.

Ação executada: Conferi o status e a asserção do teste de criação.

Evidência: Criação válida retornou HTTP 201; `test_fluxo_criar_consultar_atualizar_excluir` verifica `criada.status_code == 201`.

Arquivo e função: `backend/app/main.py`, decorator de `criar_reserva`: `status_code=status.HTTP_201_CREATED`; `tests/test_api.py`: `test_fluxo_criar_consultar_atualizar_excluir`.

Explicação: 201 expressa criação concluída de um novo recurso. O backend configura esse status na rota POST e o teste protege o contrato verificando o código recebido pelo cliente.

8. Envie `quantidade_pessoas` igual a zero pelo Swagger. Registre o `422`, identifique a informação de erro e localize a restrição correspondente no modelo.

Ação executada: Enviei POST com `quantidade_pessoas: 0` e os demais campos válidos.

Evidência: Resposta 422; erro `greater_than_equal`, localização `body.quantidade_pessoas`, mensagem “Input should be greater than or equal to 1”, valor recebido 0 e contexto `ge: 1`.

Arquivo e função: `backend/app/schemas.py`: campo `ReservaEntrada.quantidade_pessoas = Field(ge=1, le=40)`; `tests/test_api.py`: `test_rejeitar_quantidade_invalida`.

Explicação: O Pydantic rejeita o corpo antes de executar a rota porque zero fica abaixo do mínimo 1. O FastAPI converte a falha de validação em 422, com localização e regra violada no detalhe.

9. Consulte um identificador inexistente. Registre o `404`, localize a decisão no endpoint e explique por que essa situação pertence à API, embora a busca ocorra no serviço.

Ação executada: Consultei `GET /api/reservas/999999`.

Evidência: Resposta 404 com detalhe “Reserva não encontrada”.

Arquivo e função: `backend/app/main.py`: endpoint `consultar_reserva`; `backend/app/services.py`: `buscar_reserva`; teste `test_recurso_inexistente` em `tests/test_api.py`.

Explicação: O serviço consulta SQLite e devolve `None` se não encontrar linha. O endpoint decide a representação HTTP e transforma esse resultado em `HTTPException(404)`. A consulta é executada no serviço, mas o recurso inexistente é traduzido para o contrato HTTP pela API.

Parte 3 Persistência e regras

10. Crie uma reserva, encerre o servidor, reinicie e consulte novamente. Registre a evidência de permanência e localize a instrução SQL responsável pela gravação.

Ação executada: Criei a reserva id 1, encerrei e reiniciei o processo Uvicorn na porta 8001 e consultei `GET /api/reservas/1`.

Evidência: Após o reinício, GET retornou 200 e os mesmos dados do registro id 1. O registro foi excluído ao final do exercício (DELETE 204).

Arquivo e função: `backend/app/services.py`: `criar_reserva` contém `INSERT INTO reservas`; `backend/app/database.py`: `CAMINHO_BANCO` aponta para `dados/reservas.db`.

Explicação: A tabela e os registros ficam no arquivo persistente, não na memória do processo web. Reiniciar executa novamente `CREATE TABLE IF NOT EXISTS`, sem apagar linhas. O GET após reinício confirmou a permanência.

11. Cadastre duas reservas com o mesmo laboratório, data e turno. Registre a primeira resposta e a segunda. Localize a verificação do conflito, a conversão da exceção em `409` e o teste automatizado correspondente.

Ação executada: Enviei duas criações com o mesmo laboratório, data e turno.

Evidência: Primeira tentativa: 201. Segunda tentativa: 409 com detalhe “Conflito: laboratório já reservado nessa data e turno”.

Arquivo e função: `backend/app/services.py`: `existe_conflito` e `criar_reserva`; `backend/app/main.py`: `criar_reserva` captura `ValueError` e lança 409; `tests/test_api.py`: `test_rejeitar_conflito`.

Explicação: A consulta SQL procura combinação já existente. O serviço lança `ValueError`; o endpoint converte a regra de negócio em HTTP 409. O teste automatizado verifica o status e a mensagem de conflito.

12. Altere apenas o responsável de uma reserva. Registre o `PUT`, o JSON devolvido e as instruções do frontend, endpoint e serviço que participam do fluxo.

Ação executada: Enviei PUT para `/api/reservas/1`, mantendo todos os campos e alterando apenas `responsavel` para “Responsável Atualizado”.

Evidência: Resposta 200 com JSON da reserva id 1 e o responsável atualizado; os demais campos permaneceram iguais.

Arquivo e função: `frontend/app.js`: modo edição no handler `submit` escolhe PUT; `backend/app/main.py`: `atualizar_reserva`; `backend/app/services.py`: `atualizar_reserva` executa `UPDATE`; teste CRUD em `tests/test_api.py`.

Explicação: O frontend seleciona PUT quando há id de edição e envia o JSON completo. O endpoint chama o serviço, que atualiza as colunas e busca o registro atualizado. A rota devolve 200 e o frontend mostra sucesso e atualiza a lista.

13. Exclua uma reserva e tente consultá-la. Registre `204` e depois `404`. Explique por que uma resposta `204` não possui corpo e localize o tratamento desse caso no JavaScript.

Ação executada: Excluí a reserva id 1 e consultei o mesmo id em seguida.

Evidência: DELETE retornou 204 com corpo vazio; GET posterior retornou 404. A reserva de teste da interface também foi limpa após o exercício.

Arquivo e função: `backend/app/main.py`: `excluir_reserva`; `backend/app/services.py`: `excluir_reserva`; `frontend/app.js`: `lerResposta` e `excluirReserva`; `tests/test_api.py`: teste do fluxo CRUD.

Explicação: 204 indica sucesso sem conteúdo de resposta. `lerResposta` detecta `status === 204` e retorna `null` antes de tentar ler JSON; assim não tenta interpretar um corpo inexistente. A consulta posterior confirma que o registro foi removido.

14. Use o filtro de turno. Registre a URL com query string, o resultado exibido e a montagem dinâmica da consulta SQL.

Ação executada: Consultei a lista com o filtro de turno noite.

Evidência: URL `GET /api/reservas?turno=noite`; resposta 200 com a reserva noturna id 1 antes da limpeza.

Arquivo e função: `frontend/app.js`: `listar()` monta `URLSearchParams`; `backend/app/main.py`: `listar_reservas`; `backend/app/services.py`: `listar_reservas`.

Explicação: O frontend adiciona o parâmetro escolhido à query string. O endpoint encaminha `turno` ao serviço, que acrescenta `AND turno = ?` e o parâmetro correspondente à consulta SQL, evitando concatenar o valor diretamente no SQL.

15. Abra o resumo antes e depois de criar uma reserva noturna. Registre a mudança e localize a função que calcula `total` e `por_turno`.

Ação executada: Consultei o resumo antes e depois de criar uma reserva noturna.

Evidência: Antes: total 0, noite 0. Depois: total 1, noite 1. Após excluir o registro, a base voltou ao total 0.

Arquivo e função: `backend/app/services.py`: `obter_resumo`; `backend/app/main.py`: `obter_resumo`; `frontend/app.js`: `listar()` busca `/api/reservas/resumo` e atualiza `#resumo`.

Explicação: O serviço lista registros, inicializa contadores dos três turnos e soma cada reserva, retornando `total` e `por_turno`. O frontend renderiza esses valores; a inclusão de uma reserva noturna elevou ambos os contadores correspondentes em um.

Parte 4 Testes automatizados

16. Execute `python -m pytest -v`. Registre o total de testes, o tempo e os estados apresentados. Explique como o pytest descobriu os arquivos e as funções.

Ação executada: Executei `python -m pytest -v` na cópia atividade_fs usando o Python do ambiente virtual.

Evidência: 7 testes passaram: 5 em `test_api.py` e 2 em `test_services.py`. Na execução final após restaurar a asserção, tempo 0,37 s; foi exibido um aviso de depreciação Starlette/httpx.

Arquivo e função: `pytest.ini`: `testpaths = tests`; arquivos `tests/test_api.py` e `tests/test_services.py`; funções começam com `test_`.

Explicação: O pytest lê a configuração, busca arquivos `test_*.py` em `tests` e coleta funções `test_*`. O resultado nomeia cada teste e indica PASSED; o tempo e o aviso constam no terminal.

17. Escolha o teste do fluxo CRUD. Separe no código os trechos Arrange, Act e Assert, mesmo que não estejam comentados com esses nomes.

Ação executada: Separei Arrange, Act e Assert no teste CRUD sem alterar o arquivo.

Evidência: Arrange: fixture `reserva_valida` recebida pelo teste. Act: `client.post`, `client.get`, `client.put`, `client.delete` e nova consulta. Assert: verificações de status 201/200/204/404 e campos retornados.

Arquivo e função: `tests/test_api.py`: `test_fluxo_criar_consultar_atualizar_excluir`; fixture em `tests/conftest.py`: `reserva_valida`.

Explicação: Arrange prepara dados; Act executa operações; Assert compara resposta real ao contrato esperado. A sequência cobre criação, consulta, atualização e exclusão em uma única função.

18. Explique por que `TestClient` não exige Uvicorn. Indique o caminho percorrido por uma chamada `client.post` até a resposta testada.

Ação executada: `TestClient` executa uma chamada de teste contra a aplicação ASGI em processo, sem abrir um servidor de rede.

Evidência: A suíte chamou `client.post` dentro de `with TestClient(app) as cliente`; FastAPI validou a entrada, executou endpoint e serviço, e devolveu resposta verificada pelo teste.

Arquivo e função: `tests/conftest.py`: fixture `client`; `backend/app/main.py`: `app` e `criar_reserva`; `backend/app/services.py`: `criar_reserva`.

Explicação: A chamada atravessa o transporte ASGI de teste diretamente para `app`, passa pelo roteamento FastAPI e Pydantic, endpoint e serviço, grava no SQLite temporário e volta como objeto Response. Uvicorn só é necessário para escutar requisições externas; não participa do caminho em processo do TestClient.

19. Localize `tmp_path`, `monkeypatch` e `with TestClient(app) as cliente`. Explique o papel de cada elemento e apresente uma evidência de que o banco normal não foi alterado pela suíte.

Ação executada	Localizei os três recursos na fixture e executei a suíte completa.
Evidência	`tmp_path` criou o diretório temporário; `monkeypatch` redirecionou `database.CAMINHO_BANCO` para `reservas_teste.db`; `with TestClient(app) as cliente` abriu e encerrou o contexto ASGI e o lifespan.
Arquivo e função	`tests/conftest.py`: fixture `client`; `backend/app/database.py`: variável `CAMINHO_BANCO`. Banco normal `dados/reservas.db` permaneceu com 0 reservas antes/depois da suíte; os testes usam banco temporário.
Explicação	Cada teste recebe armazenamento isolado e descartável. `monkeypatch` restaura a variável ao fim do teste; TestClient inicia e finaliza a aplicação no contexto. A contagem zero no banco normal confirma que os testes não gravaram nele.

20. Modifique temporariamente uma asserção correta para provocar uma falha. Registre a mensagem do pytest, identifique valor obtido e esperado e depois restaure o teste.

Ação executada: Troquei temporariamente a expectativa `201` por `202` em `tests/test_api.py`, executei apenas o teste CRUD e restaurei o arquivo original a partir de cópia de segurança.

Evidência: Falha provocada: `assert 201 == 202`; valor obtido 201, esperado 202. Depois da restauração, os 7 testes passaram (0,37 s).

Arquivo e função: `tests/test_api.py`: `test_fluxo_criar_consultar_atualizar_excluir`; a linha original voltou a verificar `criada.status_code == 201`.

Explicação: A falha mostra que pytest apresenta o valor real e a expectativa divergente. A restauração foi confirmada pela execução completa subsequente, que terminou com 7 passed.

21. Execute a cobertura. Informe o percentual total, escolha duas linhas não cobertas e proponha cenários de teste capazes de executá-las.

Ação executada: Executei `python -m pytest --cov=backend.app --cov-report=term-missing`.

Evidência: Cobertura total: 94%. `services.py` teve 93%; linhas não cobertas incluem 16–17 (ramo de filtro por laboratório) e 71 (atualização de id inexistente). `main.py`: 92%.

Arquivo e função: `backend/app/services.py`: `listar_reservas` e `atualizar_reserva`; `backend/app/main.py`: `abrir_interface` e ramo 404 de atualização.

Explicação: Dois cenários que elevariam cobertura: chamar GET com `?laboratorio=101` para percorrer a montagem LIKE (linhas 16–17); chamar PUT `/api/reservas/999999` para exercitar retorno None do serviço (linha 71) e 404 do endpoint. Também faltou GET `/` no teste para cobrir `abrir_interface`.

22. Compare o teste `test_rejeitar_conflito` com `test_servico_impede_conflito`. Explique a diferença entre testar pela API e testar diretamente a regra de negócio.

Ação executada: Comparei o teste via cliente HTTP e o teste direto do serviço.

Evidência: `test_rejeitar_conflito` verifica POST 201 seguido de POST 409 e detalhe HTTP; `test_servico_impede_conflito` chama `services.criar_reserva` duas vezes e espera `ValueError`.

Arquivo e função: `tests/test_api.py`: `test_rejeitar_conflito`; `tests/test_services.py`: `test_servico_impede_conflito`; regra em `backend/app/services.py`: `criar_reserva`.

Explicação: O primeiro integra validação, endpoint, tradução da exceção e resposta HTTP. O segundo isola a regra de negócio e seu erro, sem testar roteamento, JSON ou status HTTP. São camadas complementares.

Parte 5 Síntese do fluxo

23. Complete a cadeia com nomes reais do projeto: evento da interface → função JavaScript → método e rota HTTP → função do endpoint → modelo → função do serviço → SQL → código HTTP → atualização da interface.

Ação executada: Completei a cadeia com identificadores encontrados no código.

Evidência: Evento `submit` → POST `/api/reservas` → resposta 201 → mensagem de sucesso e chamada `listar()` no frontend.

Arquivo e função: `frontend/app.js`: listener `submit`, `dadosDoFormulario`, `lerResposta`, `listar`; `backend/app/main.py`: `criar_reserva`; `schemas.py`: `ReservaEntrada`; `services.py`: `criar_reserva`; `database.py`: `conectar`.

Explicação: Clique em Salvar → evento submit → `fetch` → POST `/api/reservas` → endpoint `criar_reserva(dados: ReservaEntrada)` → serviço `criar_reserva` → SQL `INSERT INTO reservas` → HTTP 201 com `ReservaSaida` → JavaScript mostra sucesso, limpa o formulário e atualiza tabela e resumo.

24. Escolha um defeito possível. Informe em qual camada ele surgiria, qual evidência o revelaria e qual teste deveria impedir sua volta.

Ação executada: Escolhi como defeito a API aceitar duas reservas para o mesmo laboratório, data e turno.

Evidência: A evidência seria duas tentativas idênticas recebendo 201, quando a segunda deveria retornar 409. No exercício observado, a primeira foi 201 e a segunda 409.

Arquivo e função: Camada: regra de negócio em `backend/app/services.py`, funções `existe_conflito` e `criar_reserva`; proteção existente: `test_rejeitar_conflito` e `test_servico_impede_conflito`.

Explicação: A regressão poderia surgir se a verificação de conflito fosse removida. O teste de serviço protege a regra diretamente e o teste da API garante que a exceção seja apresentada como HTTP 409.

Entrega

- Questionário preenchido.
- Capturas numeradas E01 a E12 conforme o roteiro de evidências.
- Arquivo de texto com comandos executados.
- Indicação de arquivo, função e trecho de código em todas as questões que solicitam localização.

Roteiro de evidências

As evidências devem mostrar somente a área necessária para comprovar o resultado. Cada imagem deve conter uma legenda curta informando ação, resultado e questão relacionada.

Código	Evidência mínima	Questões
E01	Terminal com Uvicorn iniciado	1
E02	Interface e endereço no navegador	2
E03	Swagger com as rotas disponíveis	2 e 5
E04	Criação válida com resposta 201 e JSON	6 e 7
E05	Validação rejeitada com 422	8
E06	Consulta inexistente com 404	9
E07	Registro existente após reiniciar	10
E08	Duas tentativas e resposta 409	11
E09	Atualização por PUT	12
E10	Exclusão 204 e consulta 404	13
E11	Filtro e resumo alterado	14 e 15
E12	pytest e relatório de cobertura	16 a 22

Resultados observados das evidências

Para cada evidência, informe:

1. O que foi executado.

2. O que foi observado.

3. Qual arquivo e função explicam o resultado.

4. Qual trecho confirma a explicação.

5. Qual teste protege esse comportamento ou qual teste deveria ser criado.

Registro de confronto entre execução e código

Código	Registro de execução, código e teste associado	Captura
E01	Uvicorn iniciou em http://127.0.0.1:8001; `main.py`, `app`/`lifespan`. Criar teste de inicialização se necessário.	
E02	Interface local retornou 200 e foi exibida; `main.py`, `abrir_interface`. Criar teste `client.get("/")`.	
E03	Swagger mostrou GET, POST, PUT e DELETE de `/api/reservas`; `app` gera OpenAPI. Criar verificação de `/openapi.json`.	
E04	POST de interface mostrou sucesso e POST válido retornou 201 + JSON; `app.js`, `criar_reserva`.	

	`test_fluxo_criar_consultar_atualizar_excluir`.	
E05	Quantidade 0 retornou 422 e regra `ge=1`; `schemas.py`, `ReservaEntrada`. `test_rejeitar_quantidade_invalida`.	
E06	ID 999999 retornou 404; `main.py`, `consultar_reserva`. `test_recurso_inexistente`.	
E07	ID 1 foi encontrado após reiniciar; arquivo `dados/reservas.db`, `services.criar_reserva`. Propor teste de persistência separado.	
E08	Primeiro POST 201, segundo 409; `services.existe_conflito`, endpoint `criar_reserva`. `test_rejeitar_conflito` e `test_servico_impede_conflito`.	
E09	PUT retornou 200 e JSON com responsável atualizado; `atualizar_reserva` em endpoint/serviço. Coberto pelo teste CRUD.	
E10	DELETE retornou 204 sem corpo e GET seguinte 404; `excluir_reserva`, `lerResposta`. Coberto pelo teste CRUD.	
E11	Filtro noite trouxe o registro e resumo mudou de 0 para 1 à noite; `services.listar_reservas`/`obter_resumo`. `test_filtrar_e_resumir`.	
E12	7 testes passaram; cobertura total 94%; `tests/test_api.py` e `tests/test_services.py`. A suíte e pytest-cov registram esses comportamentos.	